

11-768 HW2: Evaluating a Data-Visualization Agent

Hanmo Zhang

10-01-2026

1 Part 1: Examine Seed Agent Runs

1.1 Comparison of Ground-Truth and Validator Labels

Family	TP	TN	FP	FN	Support (+/-)	MCC
<code>execution_failure</code>	4	65	33	8	12/98	-0.0022
<code>wrong_data</code>	0	73	0	25	25/73	0.0000
<code>wrong_chart</code>	3	27	0	68	71/27	0.1096
<code>hard_to_read</code>	0	59	0	39	39/59	0.0000
Macro-MCC						0.0268

Table 1: Baseline validator on the 110 seed runs. The 12 runs labelled `execution_failure` are excluded from the other three families, which are scored over 98 runs.

As Table 1 shows, the baseline is close to chance (macro-MCC 0.027), and its disagreements with the labels fall in two places. For `execution_failure`, it flagged 37 runs, 33 of them wrongly. All 33 of those runs saved a `figure.png`. It also missed the other 8 real failures, none of which saved one. For the other three families, it almost never reports an error. `wrong_chart` returned an empty list for 68 runs, only 3 of which are clean in the ground truth, and it never predicted `wrong_data` or `hard_to_read`. These families therefore have no false positives but 132 false negatives between them. The early return makes this worse: once the validator predicts `execution_failure`, it skips all other checks.

1.2 Validator Error Type 1

Misjudged execution status. The baseline asks the model “Did the agent crash, exhaust its turns, or give up without producing a valid figure?” and treats any answer starting with “YES” as a failure. On runs 004 and 022, both of which saved a figure, the model answered “YES” and then argued the opposite. On run 022 it wrote: “The agent successfully completed the task... the resulting `figure.png` file was created and verified to exist.” The model seems to confuse the polarity of the negatively phrased question, and the parser trusts only the first word. The reverse also happens. Runs 010 and 053 saved no `figure.png` at all, yet the model answered “NO”. In every case the verdict contradicts a simple existence check on the figure.

1.3 Validator Error Type 2

Missed specification violations. The baseline asks one broad question per family, such as “Does the figure fail to follow any part of the requested chart design?”, and the model almost always answers “NO”. Many of the errors it misses are concrete and checkable. Run 009 asks for x tick labels rotated 30° , but they are horizontal and overlap. Run 013 forbids jet/rainbow/hsv but uses rainbow. Run 126 plots $(x - 4)(x - 6)(x - 8) + 60$ instead of the requested $+90$, and its y axis does not start at 0. Run 039’s script clips latitudes to $[-80, 80]$ before the Mercator transform. Run 038 draws the “Non-smoker” series in the background colour. Some of these errors are clearest in the agent’s code, and the model cannot read exact values or parameters from pixels. One vague question over a long trajectory gives a small model no concrete criteria to check, so it defaults to “NO”.

1.4 Proposed Validator Changes

For Type 1, replace the model call with a deterministic check. A run is an `execution_failure` if `figure.png` is missing or empty. On the seed set, figure presence alone matches all 12 `execution_failure` labels. The other families are checked only when a figure exists. For Type 2, persist with using LLM-as-a-judge but replace each broad question with focused, family-specific checks. Each check receives the agent’s final plotting code (extracted from the trajectory) and the figure. For `wrong_chart`, ask the model to compare the request with the code and focus on five design aspects: chart type, layout, axes, legend/colour bar/annotations, and style. For `wrong_data`, also provide the input files. Then ask about the four failure modes: wrong values, missing data, wrong selection, and wrong order. For `hard_to_read`, ask about each subtype using the figure alone: clipped text, overlapping text, elements covering data, low contrast or invisible series, indistinguishable series, and squeezed layout. Every flagged problem must come with a written observation, which discourages both blanket “NO” and blanket “YES” answers.

2 Part 2: Improve Your Validator

2.1 Implemented Changes

I rewrote `validator/solution.py` around both Part 1 proposals. *Change 1 (deterministic)*: `execution_failure` is reported if and only if `figure.png` is missing or empty. No model call is made, and the other checks run only when a figure exists. *Change 2 (model-based)*: each broad yes/no question is replaced by a focused check, and the three checks run in parallel. Each check extracts the agent’s final plotting code from the trajectory: the last command that contains `savefig`, taking the script’s text when it was written with a heredoc. It then sends that code with the figure and asks the model to fill a JSON schema that the server enforces: a short observation, then a boolean, for every item. The prompts and instructions are similar to those described above and modeled after the descriptions of the functions in the assignment. A family is reported only if at least one item is flagged with a non-empty observation, and those observations become the evidence.

2.2 Seed-Set Performance vs. Baseline

As Table 2 shows, macro-MCC rose from 0.027 to 0.540, and every family improved. `execution_failure` is now perfect. The other three families have opposite error profiles. `wrong_data` over-predicts, with 26 false positives against 4 false negatives. `wrong_chart` and `hard_to_read` under-predict:

Family	MCC		Modified validator			
	Baseline	Modified	TP	TN	FP	FN
<code>execution_failure</code>	-0.0022	1.0000	12	98	0	0
<code>wrong_data</code>	0.0000	0.4222	21	47	26	4
<code>wrong_chart</code>	0.1096	0.3970	36	25	2	35
<code>hard_to_read</code>	0.0000	0.3395	14	54	5	25
Macro-MCC	0.0268	0.5397				

Table 2: Baseline and modified validator on the 110 seed runs.

they miss 35 of 71 and 25 of 39 positives, respectively. On three runs (004, 009, 077), the model wrote an extremely long observation, hit the 2,000-token output limit, and returned truncated JSON. The runner’s retries recovered 004 and 077, but 009 needed a manual resume. This failure mode still has to be handled inside the validator.

2.3 Qualitative Analysis of Targeted Error Types

The Type 1 error is fully fixed. Type 2 is partly fixed. The checks now catch concrete violations the baseline missed: horizontal tick labels where 30° rotation was requested (009), wrong x limits (013), and the +60 formula in run 126’s code. However, they introduce a regression: `wrong_data` false positives went from 0 to 26. Two causes dominate. *False positive, run 082 (clean)*: the model estimated the “auth” median as about 100 from the pixels, compared it with 101.5, and flagged the difference. The model cannot read values precisely from pixels, and the check does not require that an error be grounded in the code. *False negative, run 038*: the “Non-smoker” series is drawn in the background colour, so the model concluded the data was missing. It reported `wrong_data` (a false positive) and missed `hard_to_read` (a false negative). Many other `wrong_data` false positives are design violations filed under the wrong family (the forbidden jet colormap in 051 and 130, the linear axis in 143). This suggests the validator needs a clearer separation between the families and an explicit “not requested” answer.

3 Part 3: Design New Evaluation Tasks

3.1 Anticipated Generalization Weaknesses

I expect two classes of task to be hard for the current validator. *Dense, precise data (dense-precise-data)* covers charts with many points or labels where correctness depends on exact values, such as dot plots, strip plots, and large scatter plots. On the seed set, the validator falsely flagged `wrong_data` on 6 of the 10 scatter- or strip-type runs that produced a figure (112, 130, 169, 196, 200, 203), often after estimating values from pixels. It also flagged a busy but readable event plot (170) as `hard_to_read`. Dense inputs can exceed the validator’s limit of 12,000 characters per file, so part of the data is cut and cannot be checked. *Multi-source joins (multi-source-join)* covers tasks that must combine two input files. None of the 110 seed runs has more than one input file, so these tasks are out of distribution. The validator shows each file separately, so the model must work out the join itself: which key links the files, what happens to unmatched rows, and how quantities such as rates are derived across files. Part 2 showed that `wrong_data` is weakest exactly when it has to reason about computations rather than read the figure.

3.2 Task Design

Task	Chart and inputs	Traps and checkable facts
<i>Class 1: dense-precise-data</i>		
<code>commute-dot-plot</code>	Dot plot of 40 states; <code>commute.csv</code>	40 crowded tick labels; exact descending order (North Dakota 37.3 at the top, Iowa 16.0 at the bottom); median line at 25.75
<code>reaction-strip-plot</code>	Jittered strip plot of 300 trials in 4 conditions; <code>trials.csv</code>	Timeouts coded 9999 must be removed first; mean bars at 432.1 / 511.9 / 587.8 / 793.3 ms; fixed condition order
<code>station-scatter</code>	500-point scatter, coloured by zone; <code>stations.csv</code> (12.5 KB)	Fixed zone colours; top 5 stations by elevation labelled (ST248, ST454, ST325, ST230, ST011); least-squares line (slope $-5.67^{\circ}\text{C}/\text{km}$)
<i>Class 2: multi-source-join</i>		
<code>regional-revenue</code>	Bars of 2024 revenue per region; <code>sales.csv</code> + <code>stores.csv</code>	Exclude 2023-12; 2 stores with no region go in an “Unassigned” bar; sorted totals North 8371.7, East 7645.3, South 3585.3, West 2584.0, Unassigned 2488.3
<code>district-case-rate</code>	Horizontal bars of 2023 cases per 100k; <code>cases.csv</code> + <code>population.csv</code>	Population is in thousands; Linden has no population and must be omitted; overall-rate line at 196.3; Glenside (380.4) at the top

Table 3: The five authored tasks.

All data is synthetic. It is generated by `scripts/make_tasks.py` with a fixed seed (7752), which also prints the answer key in Table 3. The script rejects any seed that produces ties, so every sort has exactly one correct answer. Every task contains at least one trap an agent can plausibly get wrong, so the runs yield both positive and negative labels. Every answer can be checked by a human, for example through sort order, excluded rows, or a named value. Class 1 tests whether the validator avoids calling busy but correct charts hard to read, and avoids flagging precise values it cannot verify by eye. The input for `station-scatter` also exceeds the validator’s per-file limit. Class 2 tests whether `wrong_data` can follow a join, including filters, unmatched keys, and a unit conversion.

3.3 Agent Runs and Labeling

Each task was run once with each fixed agent (Qwen2.5-Coder-3B, Ministral-3-14B, GLM-4.7-Flash) using `workflow generate`, giving 15 runs. Each agent was limited to 20 steps, 120s per shell command, and 15 minutes in total. I labelled every run by hand against the generator’s answer key: first whether a figure exists, then the plotted values, selection and order (`wrong_data`), then each explicit requirement (`wrong_chart`), then readability (`hard_to_read`). The labels are 4 `execution_failure` (all Qwen), 4 `wrong_data`, 2 `hard_to_read`, 0 `wrong_chart`, and 5 clean runs. The data errors are a reversed sort order (3 runs), dots plotted at their CSV row index under sorted labels (commute, Ministral), and ignoring that population is in thousands (district, Qwen). Both `station-scatter` runs overprint neighbouring station IDs. Two decisions were ambiguous. Three figures were saved smaller than requested because of `bbox_inches='tight'`, although their code set the requested `figsize`. I labelled these as correct, so `wrong_chart` has no positives.

3.4 Validator Performance on New Tasks

Family	TP	TN	FP	FN	Support (+/−)	MCC
<code>execution_failure</code>	4	11	0	0	4/11	1.0000
<code>wrong_data</code>	2	4	3	2	4/7	0.0690
<code>wrong_chart</code>	0	10	1	0	0/11	0.0000*
<code>hard_to_read</code>	2	9	0	0	2/9	1.0000
Macro-MCC						0.5173

Table 4: The Part 2 validator on the 15 authored runs. *Insufficient support: no `wrong_chart` positives, so MCC is 0 by convention.

Table 4 shows a macro-MCC of 0.517, but the evidence is weaker than the score. Three of the four non-execution true positives are right for the wrong reason. Both station-scatter runs are flagged for “clipped text” when the real problem is overlapping labels. `district-case-rates_ministral` is flagged for a wrong formula that is actually correct; its real error is the reversed order. Both predicted weaknesses appeared. *Dense data*: both commute order errors were missed (false negatives). The dot plots look tidy, and the model cannot check 40 values against a sort order by eye. Contrary to my prediction, busy but correct plots were not flagged as hard to read. *Multi-source joins*: both correct `regional-revenue` runs were flagged as `wrong_data` (false positives). The model re-added about 50 monthly values in its head and claimed totals such as 11,150.2 against the true 8371.7. Known problems still occur – no matter how much instructions are given to the judges, some mistakes are still missed or false positives raised.

3.5 Harbor Packaging

I scaffolded all five tasks with `workflow package`; each asks for `plot.py` and a `plotted.values.json` with a fixed schema, such as bar labels in on-screen order plus values. The verifier checks that the PNG is real, that the inputs’ sha256 are unchanged, that each sidecar value matches a hand-derived literal (`test_s2_sidecar_value_matches_the_answer[key]`), and that re-running `plot.py` reproduces the same sidecar and pixels. All tasks pass `preflight.py`. With `harbor run`, every task scores **1.0** with `-a oracle` and **0.0** with `-a nop`; the course example likewise gives 1.0 (oracle), 0.0 (nop) and 0.0 for its `sum_crates` mutant. The `thousands_unit` mutant (district) ignores that population is in thousands. It scores **0**, rejected by `...[rates]` and `...[overall_rate]`. The `overlapping_labels` mutant (station) plots correct data but overprints neighbouring station IDs, as the GLM and Ministral agents did. It scores **1.0**, because no check reads rendered text. Table 5 shows which families such a verifier can decide. It cannot grade the private runs: its answer key is a set of literals for one known task, it relies on outputs the private runs never produced, and it cannot judge the families that depend on the rendered image at all.

4 Part 4: Iterate on Your Design

4.1 Additional Improvement Motivated by Part 3

An error analysis after Part 3 showed that the model fails on facts read from pixels. On the seed set, the chart check missed 6 of 7 missing axis labels, 5 of 7 exact-text errors and 9 of 13 colormap errors. The readability check missed 14 of 15 colour problems: low-contrast text, invisible series

Family	Can a deterministic per-task verifier decide it?
<code>execution_failure</code>	Yes: the file exists and is a valid, non-blank PNG.
<code>wrong_data</code>	Largely yes: compare the sidecar with hand-derived literals. It cannot prove that the sidecar describes what is on screen, for example a reversed axis.
<code>wrong_chart</code>	Partly: structural facts (scale, limits, title text, figure size) can be read from a matplotlib manifest, but only for requirements written into the test, and not for non-matplotlib output.
<code>hard_to_read</code>	No: overlap, clipping and contrast are properties of the rendered image. The <code>overlapping_labels</code> mutant passes every check.

Table 5: Which error families a deterministic Harbor verifier can decide.

and indistinguishable series. These facts exist exactly in matplotlib, so I re-run the agent’s final code with matplotlib hooked. The hook records every drawn text’s position and colour, each axes’ titles, labels, scales, ticks, spines, gridlines, legend, colorbar and projection, each series’ colour, line style and marker, and every colormap called. Deterministic checks then decide explicitly phrased requirements (exact title or label text, legend, colorbar, forbidden or named colormaps, log scale, tick rotation, removed ticks, hidden spines, gridlines, 3D or polar, named series colours, text outside the image) and four readability subtypes: overlapping text, clipped text, text contrast below 2.5:1 against the pixels behind it, and invisible or indistinguishable series. These results override the model, which keeps the remaining judgements and checks everything when the code cannot be run due to dependency issues (10 of 109 figures). `wrong_data` first reads the code, then uses the figure to verify its finding. This does risk over-fitting on the seed set. However, I believe that this risk is not very high – most real graphing tasks will have a similar structure and requirements to the seed set, and when the deterministic check fails, I still fall back onto the model for a verdict.

4.2 Final Evaluation on Seed and New Tasks

Family		Seed (110 runs)					Authored (15 runs)				
		TP	TN	FP	FN	MCC	TP	TN	FP	FN	MCC
<code>execution_failure</code>	Before	12	98	0	0	1.0000	4	11	0	0	1.0000
	After	12	98	0	0	1.0000	4	11	0	0	1.0000
<code>wrong_data</code>	Before	21	47	26	4	0.4222	2	4	3	2	0.0690
	After	20	49	24	5	0.4130	2	6	1	2	0.3858
<code>wrong_chart</code>	Before	36	25	2	35	0.3970	0	10	1	0	0.0000*
	After	46	27	0	25	0.5800	0	11	0	0	0.0000*
<code>hard_to_read</code>	Before	14	54	5	25	0.3395	2	9	0	0	1.0000
	After	27	56	3	12	0.6813	2	9	0	0	1.0000
Macro-MCC	Before										
	After										

Table 6: Confusion counts and MCC before (Part 2 validator) and after (final validator). *No `wrong_chart` positives in the authored labels, so MCC is 0 by convention.

As Table 6 shows, seed macro-MCC rose from 0.540 to 0.669, and authored macro-MCC from 0.517 to 0.596. The gains come from the deterministic checks. `wrong_chart` rose from 0.397 to

0.580, with 10 more true positives and no false positives on either set. `hard_to_read` doubled, from 0.340 to 0.681, and now rests on real detections rather than Part 2’s hallucinated clipping. On the 99 re-runnable figures, the deterministic checks raised 72 flags, and only one was wrong: a red annotation on a red bar (136) that the label does not count. `wrong_data` barely moved (0.422 $\rightarrow$ 0.413). It catches 20 of 25 data errors but has 24 false positives, and 22 of those are runs whose real error is `wrong_chart`: the model files design problems such as “jet instead of viridis” (130) or “a 4x1 grid instead of 2x2” (131) as data.

4.3 Qualitative Analysis of Self-Authored Tasks

On the authored runs, the station-scatter overlaps are now reported with the correct labels (“‘ST454’ and ‘ST325’ overlap”), and both `regional-revenue` runs are judged correct. *False negative*, `commute-dot-plot_glm`: the code runs `df.sort_values('minutes', ascending=False)`, then `y_positions = range(len(df_sorted))` and `ax.scatter(..., y_positions)` without `invert_yaxis()`, so the longest commute is drawn at the bottom. The prompt names this exact trap, yet the model answers “ok”. I hypothesise that a small model cannot simulate matplotlib’s coordinate convention, so order needs a deterministic check. *False positive*, `reaction-strip-plot_ministral`: the validator claims that the means include the 9999 timeouts because “the code uses `df['rt_ms']` directly”. But the code’s first step is `df = df[df['rt_ms'] != 9999]`, and every mean is computed from that filtered `df`. I hypothesise that the model does not track the reassignment of `df`, so a filter applied earlier in the script is invisible to it.

4.4 Further Changes

I made four further changes. (1) A deterministic DPI and size check reads the PNG metadata and skips `bbox_inches='tight'` saves. It flags exactly the four labelled seed runs. (2) When the code cannot be re-run, a pixel check keeps a model “clipped text” claim only if something drawn touches the image edge. (3) Robust JSON: the model sometimes looped on whitespace until it reached the token limit. Each observation is now capped at 300 characters, generation stops at the first blank line, and cut-off answers are repaired. (4) I tried, then removed, a filter that dropped `wrong_data` findings mentioning design terms. It cut false positives on a 20-run sample, but on the full seed set it dropped 9 real errors. Intermediate versions scored 0.488 (code-only `wrong_data`) and 0.547 (with that filter) on the seed runs. The final validator executes agent-written code in a temporary directory, with a 60s timeout and a model fallback on any failure; the assignment does not say whether this is allowed. Seaborn and scipy are absent from the validator environment, so scripts that import them always fall back to the model.